



MÓDULO PROFESIONAL 0612 · ENTORNO CLIENTE

Arquitecturas y Tecnologías de Programación sobre Clientes Web

Guía didáctica y conceptual completa del Resultado de Aprendizaje 1 (RA1). Análisis accesible de modelos cliente/servidor, motores del navegador, Event Loop, TypeScript, WebAssembly, Sandbox y herramientas de auditoría web.

⚡ Criterios Oficiales a) al f)

🌐 Arquitectura Cliente / Servidor

🚀 Motor V8 y Compilación JIT

🔄 Event Loop y Asincronía

🛡️ Seguridad Sandbox & SOP

📊 Auditorías con Google Lighthouse

RESULTADO DE APRENDIZAJE 1 (RA1)

Los 7 Pilares de la Programación en Cliente Web

01

Cliente/Servidor y SPAs

Reparto de tareas entre navegador y backend, y evolución a aplicaciones de una sola página.

02

Anatomía y Motor V8

Motores visuales vs motores JS y compilación en tiempo real (JIT) para máxima velocidad.

03

Asincronía y Event Loop

El modelo monohilo de JS, ayuda del navegador (Web APIs) y coordinación de tareas.

04

JS, TypeScript y Wasm

El estándar universal (JS), tipado estático previo (TS) y cómputo binario potente (Wasm).

05

Sandbox y Mismo Origen

El aislamiento de seguridad para proteger tu disco duro y las reglas SOP / CORS.

06

Carga HTML y Módulos ES6

Scripts síncronos vs async vs defer, y organización modular nativa moderna.

07

DevTools y Google Lighthouse

Pestañas clave de depuración (Elements, Console, Sources) y el boletín de notas de 0 a 100 con Lighthouse.

MODELOS DE ARQUITECTURA WEB

Comunicación Cliente/Servidor y SPAs

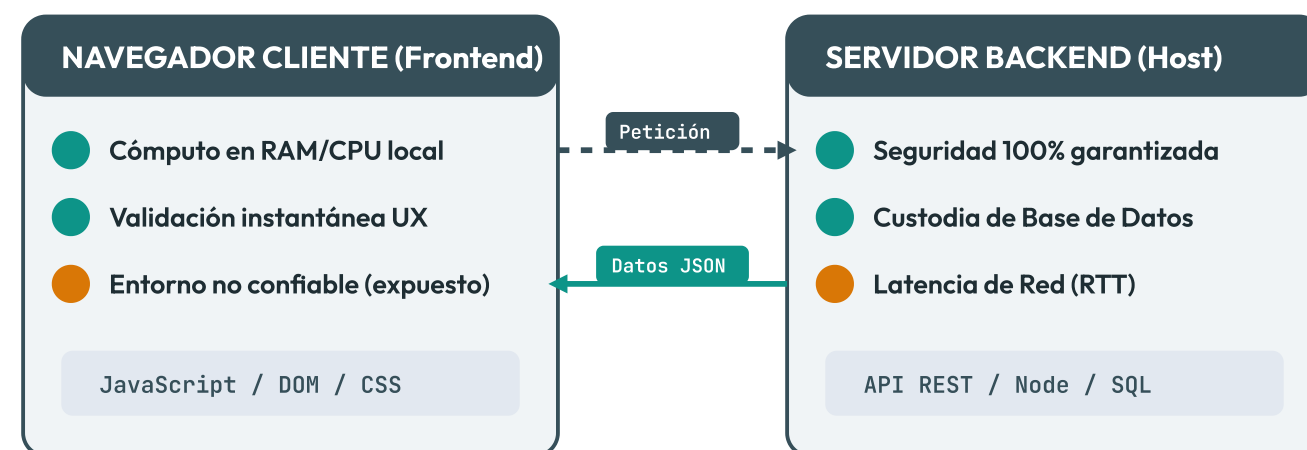
El Servidor (Backend): La Caja Fuerte

Almacena y gestiona la base de datos de forma segura. **Es el único entorno donde se garantiza la seguridad definitiva** y las reglas de negocio privadas. Nadie desde fuera puede ver ni alterar su código.

El Cliente (Frontend): Rapidez y Experiencia

Se ejecuta en el ordenador o móvil del usuario. Dibuja los botones y pantallas, y **valida formularios al instante sin latencia de red** para dar feedback inmediato al usuario.

Pregunta Clásica de Examen: ¿Por qué validar en el cliente si es obligatorio validar en el servidor? *Validar en el cliente ofrece una experiencia rápida (UX), pero nunca sustituye la seguridad obligatoria del backend, ya que el código del navegador puede ser modificado por cualquier usuario.*



EVOLUCIÓN ARQUITECTÓNICA: TRADICIONAL VS SPA

1. Web Clásica (Multi-Page)

- Cada clic descarga una página completa.
- Pantallazos en blanco en transiciones.
- Alto tráfico redundante de HTML/CSS.

⚠ Recargas completas del navegador

2. Single-Page Applications (SPA)

- Carga un único cascarón inicial.
- Navegación fluida sin recargar pantalla.
- Solo intercambia datos puros (JSON).

✓ Experiencia fluida tipo aplicación

ARQUITECTURA DEL NAVEGADOR

Motores del Navegador y Compilación JIT



Motor de Renderizado vs Motor JavaScript

El navegador tiene dos mitades trabajando en equipo:

- **Motor de Renderizado** (Blink, Gecko, WebKit): Lee HTML y CSS para dibujar los píxeles y calcular el diseño.
- **Motor de JavaScript** (V8, SpiderMonkey, Nitro): Ejecuta la lógica del código.



Compilación en Tiempo Real (JIT - Just-In-Time)

Para no ser lento, el motor **V8** no interpreta línea por línea: empieza a ejecutar de inmediato y compila directamente a **código máquina ultrarrápido** aquellas funciones que se llaman con frecuencia.

Consejo Práctico: Escribe código predecible y ordenado. Si una función espera números, no le pases textos mezclados; así el motor JIT puede mantener tu código a máxima velocidad.

LOS GRANDES NAVEGADORES Y SUS MOTORES

Chrome / Edge

Render: **Blink**

Motor JS: **V8 (Google)**

Usado en Node.js

Mozilla Firefox

Render: **Gecko**

Motor JS: **SpiderMonkey**

1er motor histórico

Apple Safari

Render: **WebKit**

Motor JS: **Nitro (JSC)**

iOS, iPadOS, Mac

EL PIPELINE JIT DE V8: DE TEXTO A CÓDIGO MÁQUINA

1

Código JS

Texto plano
archivo .js

2

Ignition

Intérprete rápido
sin esperas

3

Perfilado

Detecta código
repetido ("hot")

4

TurboFan

Código máquina
optimizado

¿Por qué es revolucionario el compilador JIT?

Permite que aplicaciones web complejas (Google Maps, hojas de cálculo, juegos) se ejecuten a velocidad nativa en la CPU sin obligar al usuario a instalar nada.

CONCURRENCIA EN JAVASCRIPT

Asincronía y el Event Loop

La Analogía del Cocinero Monohilo

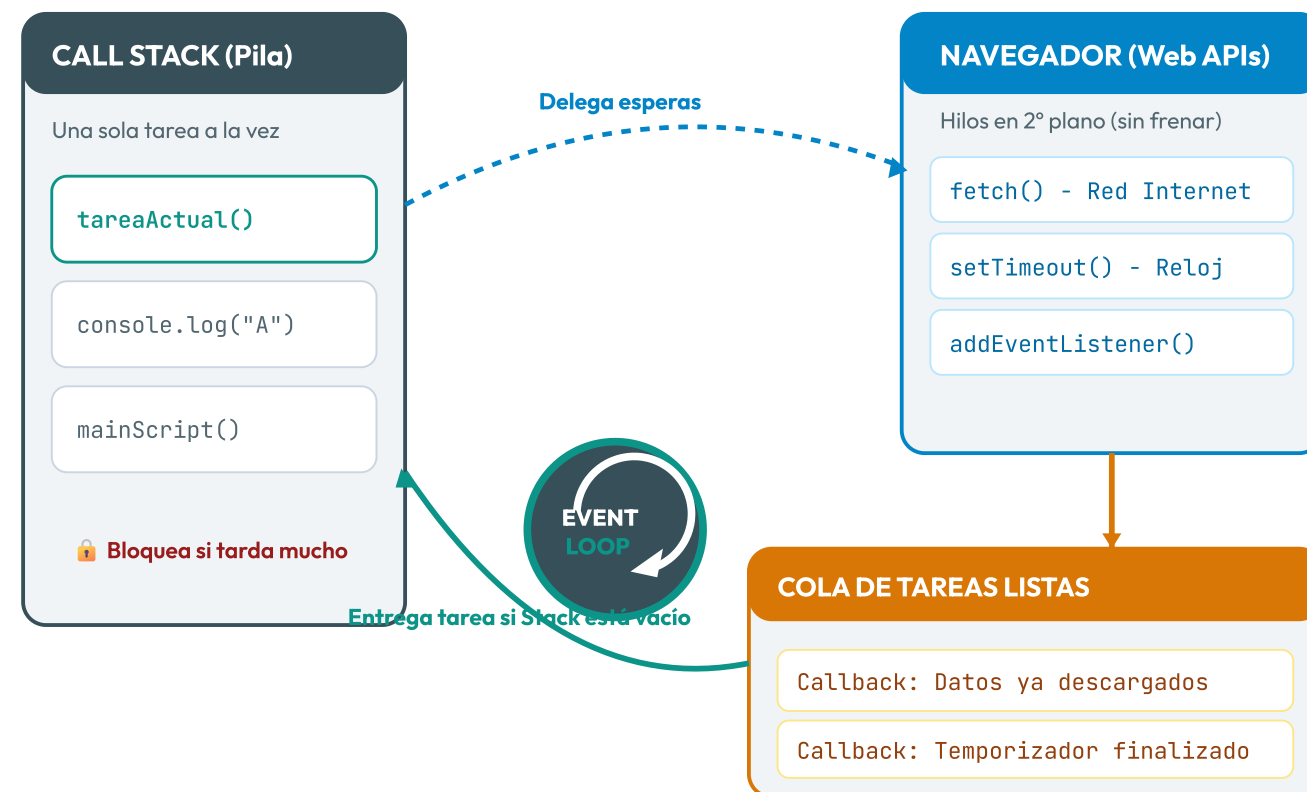
JavaScript es **Single-Thread** (un solo hilo): es como un cocinero con una única sartén. Si se quedara esperando 10 minutos a que hierva el agua, la cocina entera se congelaría.

Solución: Delega las esperas en el navegador (Web APIs).

El Event Loop: El Coordinador de Tareas

El **Event Loop** vigila continuamente: en cuanto el código principal termina y la sartén queda libre, toma la siguiente tarea lista de la cola de espera (ej: datos de internet o temporizador cumplido) y se la entrega a JavaScript.

Pregunta Estrella de Exámenes FP: ¿Por qué un `setTimeout(fn, 0)` no sale el primero? Porque pasa a la cola de espera del navegador, y el *Event Loop* espera a que todo el código principal termine antes de dejarlo pasar.



¿CÓMO PINTA LA PANTALLA EL NAVEGADOR? (3 PASOS CLAVE)

1. Estructura y Estilos

Lee HTML y reglas CSS



2. Posiciones (Layout)

Calcula medidas y cajas



3. Pintado (Paint)

Dibuja los píxeles reales

LENGUAJES EN EL CLIENTE

JavaScript, TypeScript y WebAssembly

● JavaScript: El Estándar Universal

El único lenguaje de programación que **todos los navegadores entienden de forma nativa sin plugins**. Regulado por el estándar ECMAScript con mejoras anuales constantes (`let/const`, `async/await`).

◆ TypeScript: Prevención de Errores en Proyectos

Añade tipos estáticos a JavaScript. Te avisa de fallos en el editor mientras programas.

Clave de examen: El navegador no entiende TypeScript; se debe compilar (transpilar) a JavaScript estándar antes de publicar.

¿Qué es WebAssembly (Wasm)?: Es un formato binario para ejecutar código en lenguajes como C++ o Rust a velocidad casi nativa. No sustituye a JavaScript; se usan juntos en herramientas como **Figma o videojuegos 3D**.

JAVASCRIPT

ESTÁNDAR NATIVO

- Ejecución nativa directa
- Manipulación total DOM
- Eventos del usuario
- Peticiones Fetch / AJAX
- Tipado dinámico flexible
- **El corazón de la web**

Rol Principal:
Interactividad y UI

TYPESCRIPT

TIPADO ESTÁTICO

- Superset oficial de JS
- Tipos en el editor
- Detección previa de bugs
- Interfaces y contratos
- **Se transpila a JS limpio**
- Cero coste en runtime

Rol Principal:
Seguridad en Desarrollo

WEBASSEMBLY

CÓDIGO BINARIO

- Velocidad cuasi-nativa
- Creado en C++, Rust, Go
- Cómputo matemático
- Motores de juegos 3D
- Edición gráfica (Figma)
- **Complementa a JS**

Rol Principal:
Máximo Rendimiento

SEGURIDAD EN EL NAVEGADOR

El Sandbox y la Regla del Mismo Origen



El Sandbox (Caja de Arena)

Al entrar en una web descargas código de desconocidos. El navegador lo encierra en una **jaula protegida**:

- **Prohibido tocar archivos del disco duro** sin permiso explícito.
- **Permisos obligatorios** para usar la cámara, micrófono o ubicación GPS.



La Regla del Mismo Origen (SOP) y CORS

Una web solo puede pedir datos a su propio servidor si coinciden 3 datos: **Protocolo + Dominio + Puerto**.

Si necesitas consultar una API externa legítima, esa API debe autorizarte expresamente mediante la cabecera **CORS**.

Petición Preflight (OPTIONS): Antes de enviar peticiones importantes o destructivas a otro servidor, el navegador envía una consulta previa automática con **OPTIONS** preguntando si tiene permiso.

EL ENTORNO AISLADO SANDBOX (CAJA DE ARENA)

DENTRO DEL NAVEGADOR

- ✓ Ejecuta JavaScript
- ✓ Modifica el DOM y estilos
- ✓ Guarda en localStorage local

Entorno aislado y vigilado

X

SISTEMA DEL USUARIO

- ⊖ Disco duro C:\ privado
- ⊖ Cámara web (sin permiso)
- ⊖ Memoria física del S.O.

Protección total contra malware

SAME-ORIGIN POLICY: LA REGLA DE LAS 3 COINCIDENCIAS

ORIGEN BASE: `https://tienda.es:443`

`https://tienda.es:443/productos.html`

✓ MISMO ORIGEN

`http://tienda.es:443`

X Protocolo Distinto

`https://blog.tienda.es:443`

X Subdominio Distinto

INTEGRACIÓN DE CÓDIGO

Carga de Scripts en HTML y Módulos ES6



¿Cómo cargar los scripts? defer es la clave

- **Script normal:** Bloquea la lectura del HTML mientras descarga.
- **async:** Descarga en paralelo, pero se ejecuta en cuanto llega sin respetar el orden.
- **defer (Recomendado):** Descarga en paralelo y se ejecuta en orden cuando todo el HTML está listo.



Módulos Modernos (`type="module"`)

Permiten dividir el proyecto en archivos limpios conectándolos con `import` y `export`. Tienen comportamiento `defer` automático y sus variables están aisladas sin contaminar el objeto global `window`.

Buena Práctica Profesional: Nunca uses manejadores en línea (`<button onclick="...">`). Enlaza siempre los eventos con `addEventListener` desde tu archivo JavaScript.

COMPARATIVA DE CARGA DE SCRIPTS EN EL NAVEGADOR

1. Script Clásico: `<script src="app.js">`

Detiene el dibujo del HTML mientras descarga y ejecuta

Lee HTML...

Descarga y Ejecuta JS

Continúa leyendo HTML...

2. Script Asíncrono: `<script src="analitica.js" async>`

Descarga en paralelo, pero se ejecuta en cuanto llega pausando el HTML

Lee HTML en paralelo

Ejecuta JS

Descarga en 2º plano

3. Script Diferido: `<script src="app.js" defer>` ★ RECOMENDADO

Descarga en paralelo y se ejecuta justo al terminar el HTML en orden

Lee todo el HTML sin pausas

✓ Ejecuta ordenado

Descarga en paralelo sin bloqueos

MÓDULOS NATIVOS (`<script type="module" src="main.js">`)

Comportamiento diferido (defer) automático y aislamiento modular de variables. Las variables declaradas no contaminan el objeto global window.

```
export function suma(a, b) { ... } —> import { suma } from './calc.js'
```

HERRAMIENTAS DE DESARROLLO

DevTools y Google Lighthouse

Las 4 Pestañas Clave de DevTools (F12)

- **Elements:** Inspecciona y retoca el HTML y CSS en tiempo real.
- **Console:** Muestra mensajes de error y pruebas con `console.table()`.
- **Sources:** Pausa el código paso a paso con puntos de parada o `debugger;`.
- **Network:** Registra archivos descargados y tiempos de respuesta.

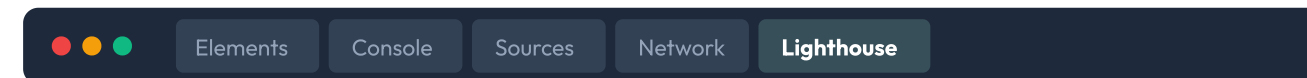
Google Lighthouse: El Boletín de Notas

Auditoría automática que califica tu web de **0 a 100** en 4 áreas:

1. **Rendimiento** (velocidad) · 2. **Accesibilidad** (para todos)
3. **Buenas Prácticas** (seguridad) · 4. **SEO** (Google)

3 Claves para Sobresaliente en Lighthouse:

1. Optimiza y comprime las imágenes (WebP).
2. Pon siempre el atributo `alt="..."` en las fotos.
3. Usa etiquetas semánticas (`<header>`, `<nav>`, `<main>`).



AUDITORÍA LIGHTHOUSE (PUNTUACIONES 0 - 100)



Rendimiento

Carga ultrarrápida
sin tirones visuales



Accesibilidad

Contraste y apoyo
para discapacidades



Buenas Prácticas

Conexión HTTPS y
código seguro moderno



SEO

Visibilidad alta
en buscadores Google

HERRAMIENTAS COMPLEMENTARIAS EN EL EDITOR (VS CODE)

ESLint (El Corrector)

Detecta fallos de sintaxis y
variables no usadas mientras
escribes tu código fuente.

Prettier (El Formateador)

Aplica sangrías, comillas
consistentes y saltos de línea
automáticos al guardar.



RESUMEN PARA EXPOSICIÓN Y EXAMEN OFICIAL

Las 4 Preguntas Imprescindibles que Debes Dominar

1. ¿Dónde se valida de forma segura?

En el servidor (backend). En el cliente se valida por rapidez y experiencia de usuario (UX).

2. ¿Qué motor JS usan Chrome y Edge?

El motor V8 de Google, que utiliza compilación Just-In-Time (JIT) a código máquina.

3. ¿Por qué JS no congela la pantalla?

Porque es monohilo pero delega las esperas en el navegador (Web APIs), coordinadas por el Event Loop.

4. ¿Qué requiere Same-Origin Policy?

Coincidencia exacta en la tupla: Protocolo + Dominio + Puerto. Para excepciones se usa CORS.